

Atividade Pratica - DevOps

Plataforma fullstack para gestao de clientes, favoritos e compras, com frontend web, API REST, Integracao com Fake Store API, autenticao, painel administrativo, persistencia em Firebase Realtime Database, containerizacao e CI/CD.

Visao geral

O projeto foi construido para cobrir um fluxo completo de produto:

- autenticao e sessao de usuario
- gestao de clientes
- catalogo de produtos com filtros
- favoritos/carrinho
- compra com PIX e QR Code
- administracao de usuarios e produtos
- persistencia em banco realtime
- deploy e pipeline automatizada

Stack tecnica

- Frontend: HTML, CSS, JavaScript (vanilla)
- Backend: Node.js, TypeScript, Hono
- Banco principal: Firebase Realtime Database
- Auth: Firebase Auth (com fallback local)
- API externa: Fake Store API
- Containerizacao: Docker e Docker Compose
- CI/CD: GitHub Actions
- Deploy alvo: Cloudflare Workers (backend), Cloudflare Pages/Netlify (frontend)

Funcionalidades completas

1. Identidade visual e carregamento inicial

- Logo aplicada nas telas principais (login e app)
- Nome do produto exibido na interface
- Favicon configurado
- Metadados de compartilhamento (Open Graph e Twitter Card)

2. Autenticacao e contas

- Login de usuario
- Cadastro de nova conta
- Integracao com Firebase Auth
- Fallback local quando auth remoto nao esta disponivel
- Controle de ~~sessao~~ com persistencia local
- Logout com tentativa de encerramento de ~~sessao~~ remota

3. Termo de responsabilidade

- Checkbox obrigatorio no login
- Modal com leitura do termo
- Bloqueio de acesso enquanto nao houver aceite

4. Perfil e navegacao da aplicacao

- Sidebar com dados do usuario autenticado
- Contador de favoritos/carrinho
- Navegacao entre secoes:
 - catalogo
 - carrinho
 - pagamento
 - painel admin

5. Catalogo de produtos

- Consumo da Fake Store API
- Renderizacao de cards de produtos
- Busca por texto
- Filtro por categoria
- Traducao de categorias para pt-BR
- Atualizacao visual em tempo real conforme filtros

6. Favoritos e carrinho

- Favoritar produto a partir do catalogo
- Remover dos favoritos
- Evita duplicidade no carrinho
- Pagina dedicada de carrinho
- Persistencia local do carrinho
- Sincronizacao de favoritos por cliente via backend

7. Pagamento via PIX

- Geracao de chave/transacao dinamica

- Geracao de payload BR Code (EMV)
- Calculo de CRC16 para payload
- Exibicao de QR Code para pagamento
- Copia da chave/payload
- Registro de compra no backend apos iniciar pagamento

8. Gestao de clientes (CRUD)

- Criar cliente
- Listar clientes
- Editar cliente
- Excluir cliente
- Regra de e-mail unico
- Mensagens de erro/sucesso por operacao

9. Painel administrativo

- Acesso controlado por usuario admin
- Abertura via botao dedicado (engrenagem)
- Gestao de usuarios:
 - editar nome, email e senha local
 - excluir usuario
- Gestao de produtos:
 - editar titulo, preco, categoria, imagem, descricao
 - editar rating
 - salvar override de produto

10. Persistencia de dados

- Clientes salvos no Firebase Realtime Database
- Favoritos salvos no Firebase Realtime Database
- Compras salvas no Firebase Realtime Database
- Leitura sincronizada do backend para o frontend
- Fallback para armazenamento em memoria (backend) e localStorage (frontend) quando necessario

Arquitetura de persistencia

Backend

- Repositorio por interface (ClientRepository)
- Implementacoes:
 - FirebaseRepository (producao)
 - InMemoryRepository (fallback/dev)
- Se FIREBASE_DATABASE_URL e FIREBASE_SERVICE_ACCOUNT_JSON estiverem definidos, usa Firebase automaticamente.

Frontend

- Sessao, caminho e configuracoes locais em localStorage
- Cadastro e leitura sincronizada com endpoints do backend
- Uso de Firebase Auth para autenticacao quando disponivel

Endpoints da API

Health

- GET /health

Clientes

- POST /api/clients
- GET /api/clients
- PUT /api/clients/:id
- DELETE /api/clients/:id

Favoritos

- POST /api/clients/:id/favorites
- GET /api/clients/:id/favorites
- DELETE /api/clients/:id/favorites/:productId

Compras

- POST /api/clients/:id/purchases
- GET /api/clients/:id/purchases

Estrutura de pastas

- backend/: API REST, repositorios, configuracao de runtime/deploy
- frontend/: aplicacao web, estilos, scripts e assets
- .github/workflows/: CI e automacoes de deploy
- docker-compose.yml: orquestracao local dos servicos

Executar localmente

Opcao 1: Docker Compose

1. Na raiz do projeto:

```
docker compose up --build
```

2. Endereços:

- Frontend: `http://localhost:8080`
- Backend: `http://localhost:3000`
- Healthcheck: `http://localhost:3000/health`

Opcao 2: Sem Docker

Backend:

1. Entre em `backend/`

2. Instale dependencias:

```
npm install
```

3. Rode em desenvolvimento:

```
npm run dev
```

4. Build e start:

```
npm run build
```

```
npm start
```

Frontend:

- Servir pasta `frontend/` com servidor estatico (exemplo: `http-server`) e apontar URL da API no app.

Configuracao Firebase

Arquivo de exemplo: `backend/.env.example`

Arquivo real: `backend/.env`

Variaveis obrigatorias para Realtime Database:

- `FIREBASE_DATABASE_URL`
- `FIREBASE_SERVICE_ACCOUNT_JSON`

Variaveis adicionais:

- `PORT`
- `CORS_ORIGIN`

Observacao: o backend usa `dotenv/config`, portanto carrega o `.env` local automaticamente ao iniciar.

CI/CD

Pipeline de CI

Arquivo: `.github/workflows/ci.yml`

Etapas principais:

- instalacao de dependencias
- checagem TypeScript
- build backend
- build de imagens Docker

Pipeline de deploy

Arquivo: `.github/workflows/deploy-cloudflare.yml`

Possibilidades:

- deploy de backend no Cloudflare Workers
- deploy de frontend no Cloudflare Pages

Publicacao frontend (Netlify)

Para o frontend no Netlify, o projeto ja possui:

- `<title>` definido
- favicon com a logo
- metadados de preview (og e twitter)

Importante:

- publicar o conteudo de `frontend/`
- manter o arquivo da logo em `frontend/public/img/`
- configurar URL base da API no app apos publicar

Validacao funcional recomendada

Checklist rapido:

1. Cadastro de usuário novo
2. Login com aceite de termo
3. Criação de cliente
4. Favoritar produto
5. Abrir carrinho e comprar via PIX
6. Confirmar registro em:
 - /clients
 - /favorites
 - /purchasesno Firebase Realtime Database

Entregáveis

- Repositório GitHub: PREENCHER
- URL do frontend publicado: PREENCHER
- URL da API publicada: PREENCHER

Observações finais

- O projeto suporta modo online (Firebase) e modo fallback (local/memória).
- O backend foi estruturado para facilitar evolução para novos repositórios de dados.
- O frontend foi organizado para manter UX completa em desktop e mobile.